

Windows API — THM Cheat Sheet v3

Visual revision • Concept boxes • Definition cards • Clean study-sheet layout

01 Win32 API — The Big Picture

APPLICATION / MALWARE	WIN32 API CALL	WINDOWS DLL
FUNCTION ADDRESS	WINDOWS	IN / OUT PARAMETERS

Core Definitions

Concept	Definition
API	A set of functions/interfaces that programs use to request services from an operating system.
Win32 API	Windows programming interface containing functions used to interact with Windows components.
API Call	A specific Windows function invoked by a program.
DLL	Dynamic-Link Library containing reusable code/functions that can be loaded by processes.
Parameter	A value supplied to or returned by an API function.

02 Win32 API Structure

API	HEADERS / IMPORTS	CORE DLLs
SUPPLEMENTAL DLLs	CALL STRUCTURE	API CALL
IN / OUT PARAMETERS		

Core DLLs: KERNEL32, USER32, ADVAPI32 are common Windows libraries. Supplemental DLLs: additional libraries such as NTDLL and subsystem-specific DLLs.

03 ASLR & API Resolution

ASLR — Address Space Layout Randomization: randomizes memory locations so programs cannot safely depend on fixed addresses.

ASLR	FUNCTION ADDRESS CHANGES	PROGRAM NEEDS RESOLUTION
C / C++ → windows.h	C# / .NET → P/Invoke	

Term	Definition
windows.h	Windows header used by C/C++ programs to access Win32 API definitions.
P/Invoke	.NET mechanism for calling unmanaged functions from managed code.
DllImport	Attribute used to specify/import the unmanaged DLL containing the function.
extern	Indicates that the method implementation is outside the current C# code.
System.Runtime.InteropServices	.NET namespace containing interop/P/Invoke functionality.
IntPtr	Pointer-sized .NET type used for native pointer/handle-sized values.

04 API Naming Scheme

A ANSI / 8-bit	W Wide / Unicode	Ex Extended
----------------	------------------	-------------

A: legacy ANSI/8-bit string variant. W: wide-character/Unicode variant. Ex: extended version with additional functionality or parameters.

05 API Parameters

[in] CALLER → API	[out] API → CALLER
-------------------	--------------------

[in] means the caller supplies the value. [out] means the API produces or writes the value for the caller.

Example — WriteProcessMemory	Role
hProcess	[in] Process handle
lpBaseAddress	[in] Memory address
lpBuffer	[in] Data to write
nSize	[in] Number of bytes
lpNumberOfBytesWritten	[out] Number of bytes actually written

06 Commonly Abused APIs

API	Definition / What it does
LoadLibraryA	Loads/maps a specified DLL into the calling process.
GetUserNameA	Retrieves the current user's name.
GetComputerNameA	Retrieves the local computer name.
GetVersionExA	Retrieves OS version information.
GetModuleFileNameA	Retrieves the full path of a module/process file.
GetStartupInfoA	Retrieves STARTUPINFO information for a process.
GetModuleHandle	Returns a handle for a module mapped into the process.
GetProcAddress	Returns the address of an exported function in a DLL.
VirtualProtect	Changes memory protection for a virtual memory region.

07 Keylogger Case Study

GetCurrentProcess	GetModuleHandle	SetWindowsHookEx
WH_KEYBOARDLL = 13	KEYBOARD HOOK	UnhookWindowsHookEx

GetCurrentProcess: gets a pseudo-handle for the current process.

GetModuleHandle: gets a handle for a module mapped in the process.

SetWindowsHookEx: installs a hook into a Windows hook chain.

WH_KEYBOARDLL = 13: identifies the low-level keyboard hook type.

UnhookWindowsHookEx: removes an installed hook.

Module: an EXE or DLL loaded/mapped into a process.

Module Handle: an OS-provided handle/reference identifying that loaded module.

08 Memory — Allocation vs Protection

MEM_COMMIT 0x1000 ALLOCATION / STATE	MEM_RESET 0x00080000 ALLOCATION FLAG	PAGE_EXECUTE_READWRITE 0x40 PROTECTION
---	---	---

MEM_COMMIT: commits memory pages so they can be used by the process.

MEM_RESET: tells Windows that the contents of specified pages are no longer of interest to the caller; it is an allocation/state flag.

PAGE_EXECUTE_READWRITE: permits Read + Write + Execute on the memory region.

Rule: MEM_* answers "what happens to the memory allocation/state?" while PAGE_* answers "what operations are permitted on the memory?"

09 Shellcode Launcher Case Study

VirtualAlloc	MEM_COMMIT + RWX	Marshal.Copy
CreateThread	WaitForSingleObject	

VirtualAlloc: reserves/commits or changes the state of virtual memory pages.

Marshal.Copy: copies bytes between managed and unmanaged memory; here it copies shellcode into allocated memory.

CreateThread: creates a thread in the process's virtual address space.

WaitForSingleObject: waits for an object/thread to reach a signaled state or for timeout.

Behavior clue: VirtualAlloc + RWX + copying bytes + CreateThread is a strong indicator to investigate in-memory code execution. It is not, by itself, proof of malware.

10 Important Types & Handles

Term	Definition
Handle	An OS-managed reference used to identify an object/resource.
HWND	A handle identifying a Windows GUI window.
Module Handle	A handle/reference identifying an EXE or DLL mapped in a process.
Pointer	A value representing a memory address/reference to data or code.
IntPtr	A pointer-sized .NET value type for native pointer/handle-sized values.

11 Malware Analysis Mindset

API NAME	PARAMETERS	COMBINATION
SEQUENCE	BEHAVIOR	EVIDENCE

Golden rule: Do not decide from one API. Read API → parameters → combination → sequence → behavior.

Combination	What to investigate
SetWindowsHookEx + WHKEYBOARDLL	Keyboard monitoring / hook behavior
VirtualAlloc + PAGE_EXECUTE_READWRITE	Executable writable memory
VirtualAlloc + RWX + Marshal.Copy + CreateThread	Possible in-memory shellcode execution

12 One-Page Memory Map

A → ANSI	W → Unicode	Ex → Extended
[in] → Input	[out] → Output	ASLR → Randomized memory layout
DllImport → DLL declaration	extern → External implementation	IntPtr → Pointer-sized type
MEM_COMMIT → Commit memory	MEM_RESET → Allocation/reset flag	PAGE_EXECUTE_READWRITE → R/W/X
SetWindowsHookEx → Install hook	UnhookWindowsHookEx → Remove hook	GetModuleHandle → Module handle
VirtualAlloc → Virtual memory	CreateThread → Create thread	WaitForSingleObject → Wait